安全子系统硬件方案

1. 需求（字节）

要求描述	level	芯片必须提供的硬件能力（RTL级）	软件设计实现调用流程
固件支持数字签名	强制	Boot ROM/安全控制器集成验签状态机； SHA/SM3哈希单元； RSA/ECC/SM2验签引擎； 固件镜像只读加载路径	上电后Boot ROM读取待启动固件镜像→调用哈希/验签驱动接口校验签名→校验通过后标记镜像为可执行并跳转启动，失败则进入故障态/恢复态
不得使用已经泄漏的签名私钥签发固件	强制	RoT（Root of Trust，硬件可信根）中保存签名公钥指纹/证书版本/吊销列表摘要； 固件头包含密钥ID与版本号检查逻辑	固件发布流程在构建系统侧校验签发密钥未被吊销→启动或升级时软件读取镜像中的密钥ID/版本信息→调用RoT校验该ID是否有效后再触发验签
签名私钥存储在HSM，确保其状态可用不可见	强烈建议	芯片侧仅保留公钥/证书摘要与验签接口； 私钥不落芯片； 支持与HSM签发链匹配的证书链/Key ID校验字段	发布系统通过HSM完成签名→软件将证书链/Key ID封装进镜像元数据→芯片启动/升级时驱动读取元数据并调用验签接口完成链路校验
升级时支持基于升级可信根RTU（Root of Trust for Upgrade）的数字签名升级校验	强制	升级控制器/安全核内置RTU公钥槽； 升级包签名验签单元； 升级状态机和回滚保护计数器	管理软件下发升级包→升级服务调用RTU验签接口校验包签名和版本→通过后写入A/B分区并切换启动槽位，失败则拒绝升级
禁止存在数据通路绕过RTU的固件验签（比如在无安全启动的板卡上通过MMIO直接读写板卡flash、直接暴露板卡flash篡改固件）	强制	Flash/镜像写通路统一经过安全仲裁器； MMIO到Flash的直通写接口关闭； 内部总线访问控制/防火墙	驱动和BMC侧只通过受控升级命令写镜像→所有直接MMIO/调试写Flash请求在硬件防火墙处被拦截并记录告警
需要采取措施（包括固件签名验证，内部总线访问控制等）防止用户直接通过软件方式操作flash导致板卡无法线上恢复	强制	固件签名验证门控； Flash写保护位； 内部总线ACL； 恢复分区/只读救援镜像	普通软件无权直接写Flash→升级服务先解锁受控会话并完成签名校验→硬件开放一次性写窗口→写入完成后重新锁定并保留线上恢复路径
支持基于硬件可信根的安全启动，基于系统RoT完成安全启动过程中的链式固件验签	强制	RoT（Boot ROM + OTP/eFuse + 安全状态寄存器）； 链式验签状态机； 各级固件度量寄存器；	上电后RoT先验证一级引导→一级引导再验证后续管理核/主控固件→软件仅在前一级验证通过时才允许加载下一级镜像
验签公钥存储在OTP中，量产产品禁二次写入	强制	OTP/eFuse公钥槽； 量产锁定位； 只读公钥访问接口	制造阶段一次性烧录验签公钥→量产态软件只能读取公钥摘要用于校验与审计，不能改写公钥内容
支持安全启动验签密钥	强制	多公钥槽/密钥版本寄存器；	安全运维软件通过受控更新流程写入新的公钥

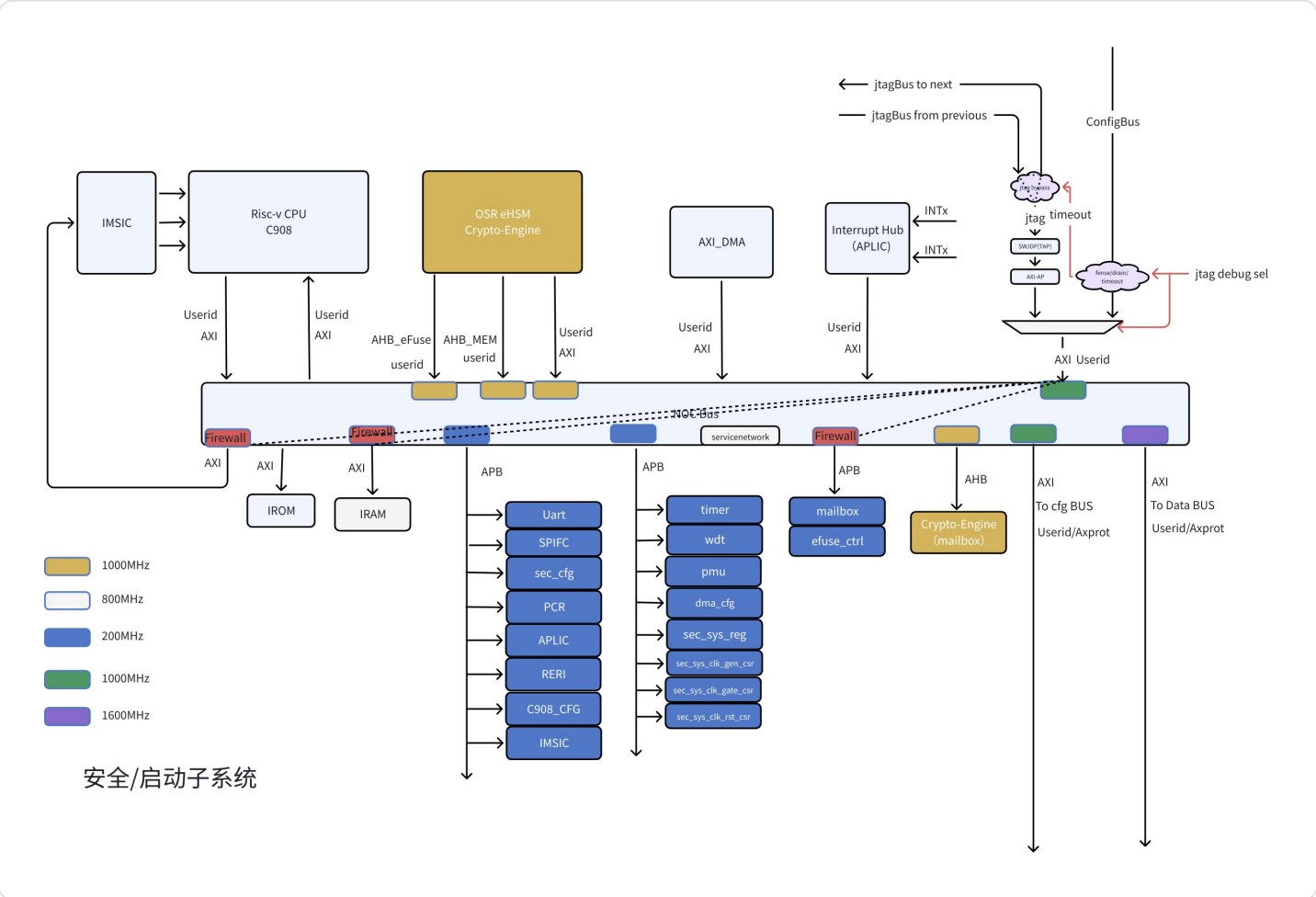
吊销及更新		吊销位图或黑名单寄存器； 安全更新公钥元数据机制	元数据/吊销列表→启动和升级时自动选择未吊销且版本有效的公钥完成验签
通过PCIe MMIO加载的各小核固件在载入板卡内存空间后必须进行固件验签	强制	PCIe下载缓冲区与板载执行存储之间的验签门控； 小核固件装载控制器； 每个小核独立的镜像状态位	Host驱动经PCIe MMIO下载小核固件到暂存区→管理固件触发验签→通过后才搬运到执行地址并释放小核复位
Option ROM需要经过完整性校验才能被加载使用	强制	Option ROM存储区完整性哈希/签名校验逻辑； 加载前门控； ROM状态寄存器	系统枚举设备时先读取Option ROM元数据→调用完整性校验接口→通过后才向主机暴露ROM内容供加载使用
支持SPDM协议，支持基于SPDM的部件度量认证	强制	SPDM硬件辅助模块； 设备证书/度量寄存器； 安全摘要/签名引擎； SPDM消息缓冲区	平台软件发起SPDM会话→固件读取度量寄存器并组装measurement block→调用签名引擎返回签名证据给验证方完成度量认证
支持基于TEE的机密计算	强烈建议	TEE安全核/隔离执行环境； 安全内存区域划分； 密钥与上下文隔离逻辑	机密作业由运行时在创建会话时申请TEE上下文→驱动将模型/密钥装入受保护内存→仅TEE内任务可访问对应数据并完成推理
内存访问控制：PCIe MMIO/DMA能访问的地址空间需要支持访问控制，与业务功能无关的内部管理逻辑（安全核、电源管理）不可被用户触达或篡改	强制	IOMMU/地址空间检查； PCIe BAR/MMIO白名单； DMA访问控制表； 管理域/业务域隔离防火墙	驱动初始化时为每类队列配置可访问地址空间→MMIO/DMA请求进入芯片后先做权限检查→非法访问直接拒绝并上报错误
功能约束：需要采取措施（包括固件签名验证，内部总线访问控制等）防止用户直接通过软件方式修改电压烧毁部件，不允许遗留绕过内存访问控制的接口	强制	电压/时钟/电源管理寄存器硬隔离； 内部总线ACL； 关键控制寄存器锁定位； 防越权写保护	普通驱动无法映射关键控制寄存器→只有受信管理固件在特权态可调用电源管理接口→所有越权写请求被硬件阻断
Option ROM所在的BAR空间不能被软件篡改	强制	Option ROM BAR区域只读映射； BAR写保护逻辑； 配置空间访问过滤	PCIe配置初始化后软件只能读取该BAR内容→任何写BAR或映射篡改请求都由配置过滤逻辑拒绝并记录日志
禁止存在绕过系统安全机制对系统进行访问的功能，禁止隐秘、不可管理、未文档化的接入认证方式	强制	调试/维护接口显式枚举； 隐藏功能熔断位； 认证访问门控； 未文档接口屏蔽逻辑	软件栈只暴露文档化管理接口→运维工具访问敏感接口必须先完成认证→未登记功能路径默认不可见不可调用
禁止存在病毒、木马等恶意软件	强制	安全启动链 + 固件白名单； 可执行镜像完整性校验； 软件供应链扫描接口/安全启动策略寄存器	发布前由CI/CD完成恶意软件扫描→部署时仅允许加载通过签名与白名单校验的镜像→运行期发现异常镜像立即阻断
禁止使用私有加密算法，用于数据加密的工作密钥不能硬编码	强制	标准加密算法硬件引擎（AES/SM4等）； 工作密钥寄存器/安全SRAM； 禁止硬编码密钥的密钥注入接口	业务软件通过KMS/安全服务下发会话密钥→驱动把密钥装入安全寄存器→数据通路调用标准算法引擎完成加解密，代码中不保存明文固定密钥
密码算法中用到的随机数需要是安全随机数	强制	TRNG/DRBG硬件随机数发生器； 随机数健康检测； 安全随机数取数接口	固件/驱动在生成工作密钥、nonce、IV时调用TRNG接口→健康检测通过后返回随机数供密码库使用
需要经过工具扫描，扫描到的高风险漏洞需要按照漏洞处理要求解决	强制	支持固件/软件物料清单导出； 安全版本寄存器； 扫描结果上报寄存器/日志接口	发布前使用SAST/DAST/SCA工具扫描固件和驱动→将高危漏洞清零后再发版→运行中可导出版本与组件信息供复核

漏洞需要及时通知字节，并跟字节对齐补丁计划	强制	补丁版本寄存器； 安全公告/告警接口； 升级兼容状态机； A/B补丁切换支持	发现漏洞后软件侧生成通报并与客户同步补丁计划→运维按计划下发补丁包→芯片通过安全升级流程完成切换并反馈状态

2. 方案架构

2.1 使用OSR eHSM的架构方案

2.1.1 安全子系统架构方案



说明：

- 基于C908 Risc-v CPU+OSR eHSM IP设计安全子系统
- 安全子系统相对封闭，其它Master无法直接访问，外部系统只能通过中断建立和安全子系统的业务联系
- 安全子系统的C908/AXI DMA和eHSM在系统中权限最高，可通过config bus和data bus访问系统中的其它空间
- 安全子系统保留debug bus的访问权限，具体访问空间及权限待定

- eHSM有自己独立的boot rom，独立封闭运行，实现各级固件的验签、解密、密钥管理、芯片生命周期管理等
- C908和eHSM通过mailbox交互，eHSM只接受来自C908的任务，其它Core或者Master没有访问通道
- sec_cfg可产生userid、axi arprot[1]/awprot[1]、ahb hnonsec信号（0为安全，1为非安全），配合firewall实现对管理子系统中的IRAM和Slave模块的访问权限管理
- 所有的镜像文件都需要通过安全子系统验签后分发给各个使用方
- 安全子系统中的SPI Flash，存储有UCIe/C2C/LPDDR等高速接口的参数，在boot阶段，管理子系统CPU需要用到，需要管理子系统CPU发中断向安全子系统中的C908申请——目的是满足字节对SPI Flash写权限只有安全子系统独享需求
- C908上电默认userid/arprot[1]/awprot[1]有效（保留最大权限），在非安全启动时（基于GPIO或者eFuse判断），C908配置service_network，把安全子系统中的firewall disable掉（不存在）
- 安全子系统中的CRM主要是负责子系统内的CRM管理及boot启动管理，在系统启动后，如果GPU发生异常，需要复位，由管理子系统负责复位重启
- User ID是否直接固化，另外user id在总线中传输时，不能被截位或者丢弃
- 主从DIE的userid需要区分

待讨论

- efuse分成两个，安全子系统和管理子系统中各放一个，其中安全子系统中存放eHSM使用的密钥信息（1KByte），管理子系统中存放其它CPU读取的内容，比如PG信息，Tempsensor等信息，这样保证安全子系统的独立性——只能开发一个控制器

两种启动方案：

1. 安全子系统通过firewall内置share memory，镜像文件host通过pcie接口可以直接写入sharememory，然后通过发消息中断通知C908
2. 安全子系统保持相对封闭，外部只能通过发中断方式通知c908（可通过IMSIC或者mailbox方式），启动时的镜像文件只能借助外部IRAM

- 上电复位后的SPI Nor Flash阶段，C908初始化PCIe后，等待Host向管理子系统中的IRAM中写入镜像，Host写入镜像后，可发中断通知C908，或者C908通过轮询方式查询IRAM中的镜像有效标志位，然后执行后面的操作
- Host通过PCIe EP的AXI Master接口写消息中断给C908

问题：

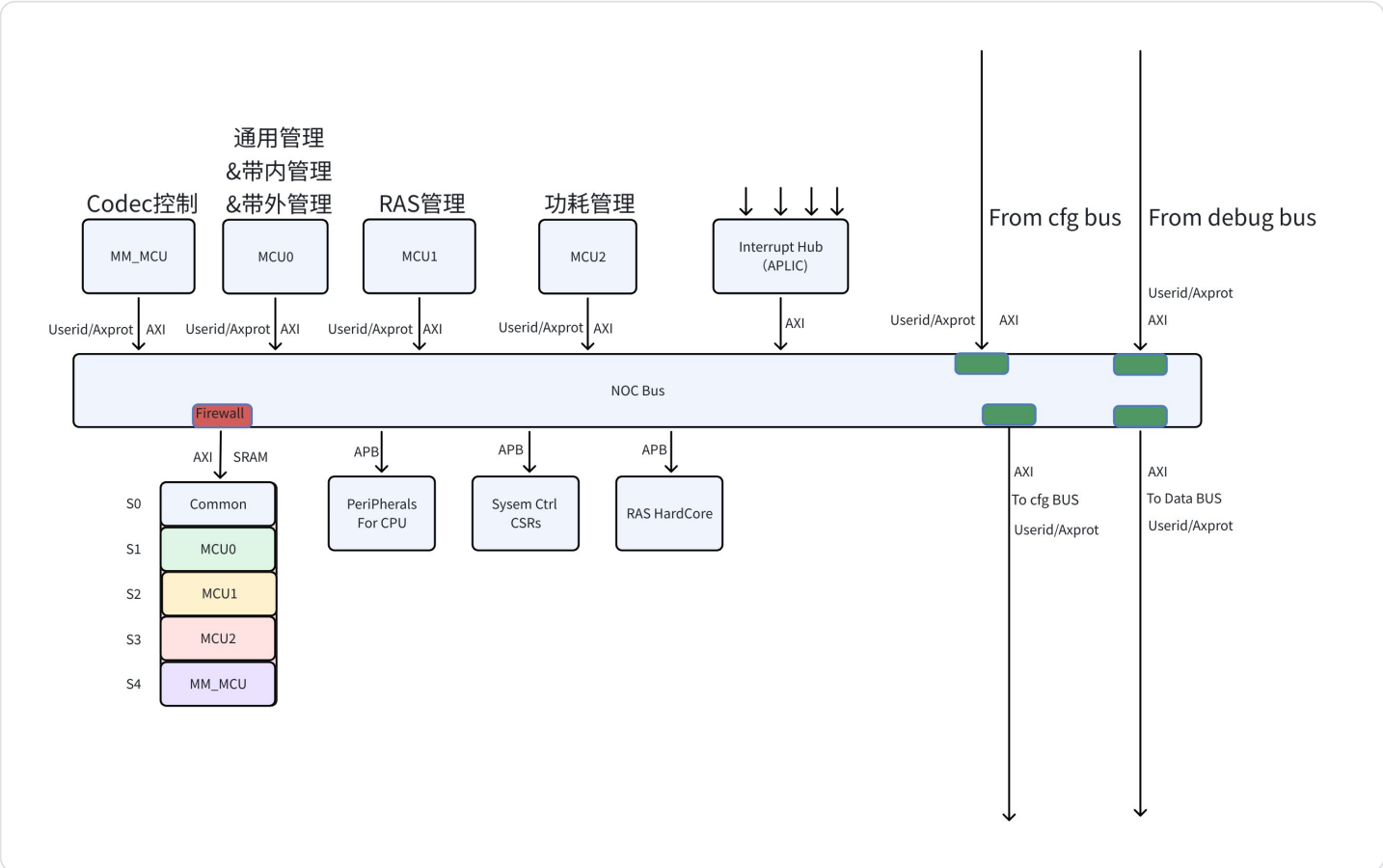
1、证书的时效性，需要用到UTC时间，eHSM UTC需要一个32K的时钟

2、eHSM需要RAM ECC功能

4、如果考虑上电UCIe的PG做boot，安全核中的C908就需要在上电时读取efuse中的PG内容，efuse是否就不需要做成2个了

5、方案选择: a、上电后安全子系统和PCIe通道复位释放，管理子系统复位先不释放，PCIe的镜像是发到安全子系统做验签和解密，验签后再打开管理子系统的复位和对应的CPU复位；b、PCIe的镜像是发到管理子系统的，所以上电后管理子系统的复位和安全子系统的复位是一起释放的；

管理子系统示意图：



MCU复位释放顺序说明：

1. 上电后安全子系统中的C908最先复位释放，然后运行SPI或者UART Boot导入的镜像文件，对PCIe进行初始化，并释放管理子系统的复位，同时hold管理子系统中各个MCU的复位；
2. 安全子系统中的C908逐次获取Host写入的各级微核的镜像文件，做完验签和解密后，上传到对应的RAM空间，并释放对应的MCU复位，如上图，会依次释放MCU0、MCU1、MCU2和MM_MCU的复位
3. 如果验签或者解密不通过，安全子系统中的C908会做好记录并上报Host/BMC

2.1.2 Userid说明

Userid的使用说明

- 通过Userid实现用户对IRAM和Slave模块的权限管理
- Userid代表Master分组或者权限，允许多个Master共用一个Userid，也可以一个master一个Userid
- Userid共4bit，表示有16组用户，每组用户对某个区域或者Slave模块的读写权限可分开设置，详见Firewall功能说明

USERID	适配Master对象	说明
4'b0000	安全子系统中的C908、eHSM和DMA Master	多个Master共用一个Userid
4'b0001	管理子系统RAS Core	
4'b0010	管理子系统中的通用&带内&带外管理MCU Core	
4'b0011	管理子系统中的Codec MCU Core	
4'b0100	管理子系统中功耗管理MCU Core	
4'b0101	Host通过PCIe访问管理子系统中的RAM	走config bus channel
4'b0110	管理子系统中的DMA权限	
4'b0111	TOP NOC中的DMA	
4'b1xxx	保留	

2.1.3 Firewall功能

Firewall是NOC的一个组件，可以放到任何一个Initiator NIU到Target NIU的链路上，一般是靠近Target NIU放置。

Firewall由WRAPPER和用户逻辑两部分构成，并通过特定的接口进行信息传递。传递给Target NIU的每一次传输，WRAPPER会提取包头信息给用户逻辑，用户逻辑根据配置寄存器，对传输的权限进行判断，并将判断结果和隐藏指示反馈给WRAPPER，针对反馈结果NOC会做出如下操作

1. 对于check pass的传输, Target NIU会将其透传给SLAVE，并由SLAVE做出 Response。
2. 对于check fail的传输，Target NIU不会将其传给SLAVE
3. 当不隐藏错误时，由Target NIU产生Response Error；

4. 当需要隐藏错误时，由Initiator NIU 产生Response OK, 此时Initiator是察觉不到访问错误的，这样就可以防止一些恶意攻击。

Firewall的用户逻辑的设计：

1. 用户逻辑在check使能的情况下，对传输做检查，否则透传。
2. 在check使能的情况下，用户逻辑会进行地址检查，安全检查和用户ID访问权限检查。只要**有一个检查是fail的，那结果就是fail的**。另外在隐藏错误使能的情况下，会送出隐藏指示信号。
3. 用户逻辑会将slave划分成若干个区间。
   - 每个区间由起始地址和结束地址划定范围。
   - 每个区间的读安全属性和写安全属性可分别配置。
   - **每个区间的userid 读权限和userid 写权限可分别配置**。
   - **每个区间可以单独使能。只有使能的区间才会进行到检查，否则该区间会被忽略掉。**
   - 一个区间可以对应一个slave，如apb salve的情况。也可以多个区间对应一个slave，如slave是sram的情况，sram最多可以被划分成8个区间，每个区间的安全属性和**用户ID访问权限可单独配置**。
4. 地址检查：若传输的起始地址落入一个使能的区间，地址检查就通过。若传输的起始地址没有落入任何一个使能的区间，地址检查就fail。
5. 安全检查：若传输的起始地址落入一个使能的区间， 就按照该区间的安全属性进行检查：
   - 区间是读安全区：标识为安全的读传输通过
   - 区间是读非安全区： 标识为安全/非安全的读传输均可通过
   - 区间是写安全区：标识为安全的写传输通过
   - 区间是写非安全区： 标识为安全/非安全的写传输均可通过
6. userid访问权限检查：
   - 所有die的master可以通过4bit的userid，最多分成16组（组0~组15）。
   - 每个区间都有16bit读权限寄存器。**每一个bit对应一个master分组**，共对应16个master分组。Bit值为1时，表示对应的master组对该区间有读权限。Bit值为0时，表示对应的master组对该区间没有读权限。例如bit[0]对应maser组0，bit[1]对应master组1，依次类推。
   - 每个区间都有16bit写权限寄存器。**每一个bit对应一个master分组**，共对应16个master分组。Bit值为1时，表示对应的master组对该区间有写权限。Bit值为0时，表示对应的master组对该区间没有写权限。例如bit[0]对应maser组0，bit[1]对应master组1，依次类推。

   如下图示例所示：假设有一笔userid等于13（4'b1101）的传输经过firewall，若该传输的地址落入区间0，因为区间0的读权限寄存器的bit[13]为1，所以读传输可以通过。因为区间0写权限寄存器的bit[13]为1，所以写传输可以通过。若该传输的地址落入区间1，因为区间1读权限寄存器的

bit[13]为1，所以读传输可以通过。因为区间1写权限寄存器的bit[13]为0，所以写传输不可以通过。

图：用户ID访问权限示例